SISTEMA DE GESTIÓN GEOMÉTRICA CON FLYWEIGHT

Patrón de Diseño: *Flyweight (Peso Mosca)*

Minimización del consumo de memoria mediante la compartición eficiente de estados intrínsecos en figuras geométricas de celdas

Autor: **Angie Nathaly Cisneros Guerrero**

20242020084

Fecha: Octubre 2026

Índice

1. Introducción **2**

2. Objetivos **3**
2.1. Objetivo General 3
2.2. Objetivos Específicos 3

3. Diagrama UML **4**

4. Explicación de las Clases **5**
4.1. Interfaz Flyweight (FiguraFlyweight.java) 5
4.2. Clase Concreta Compartida (FiguraConcreta.java) 5
4.3. Fábrica de Flyweights (FiguraFactory.java) 7
4.4. Clase No Compartida (FiguraUnshared.java) 7

5. Código de Cliente **9**
5.1. Clase Principal (Cliente.java) 9

6. Pruebas y Ejecución **11**
6.1. Ejecución en Consola 11

7. Repositorio **12**
7.1. Enlace GitHub 12

Introducción

En las aplicaciones gráficas y tableros basados en cuadrículas con múltiples elementos repetitivos (como círculos y rombos con diversas orientaciones y colores), la instanciación independiente de cada objeto consume grandes cantidades de memoria RAM de forma innecesaria.

Para dar solución a este problema de rendimiento, se implementa el patrón de diseño estructural **Flyweight**. Este patrón permite compartir objetos para soportar eficientemente grandes cantidades de granularidad fina, separando el estado intrínseco (datos compartidos e invariables como tipo, orientación y colores) del estado extrínseco (contexto dinámico como coordenadas de posición y dimensiones).

Objetivos

Objetivo General

Implementar un sistema de renderizado geométrico aplicando el patrón de diseño **Flyweight** para optimizar el almacenamiento en memoria mediante la reutilización de objetos compartidos y el manejo de instancias especializadas no compartidas.

Objetivos Específicos

- Definir una interfaz común `FiguraFlyweight` que estandarice el método de creación y dibujo de las figuras.
- Desarrollar la clase `FiguraConcreta` para encapsular el estado intrínseco invariable (tipo, orientación y colores).
- Implementar la clase `FiguraFactory` basada en listas para centralizar el almacenamiento en caché y la recuperación de objetos compartidos.
- Incorporar la clase `FiguraUnshared` para representar elementos especiales o con detalles únicos que no participan de la compartición.
- Construir la clase `Cliente` encargada de gestionar el estado extrínseco (coordenadas, ancho y alto) y demostrar el ahorro de memoria en consola.

Diagrama UML

Descripción General

El proyecto implementa una arquitectura basada en el patrón estructural **Flyweight**.
El componente central es la interfaz `FiguraFlyweight`, implementada tanto por la clase compartida `FiguraConcreta` como por la variante no compartida `FiguraUnshared`. La clase `FiguraFactory` administra una lista interna de objetos reutilizables, mientras que el `Cliente` coordina las solicitudes y suministra el contexto espacial.

Diagrama de Clases

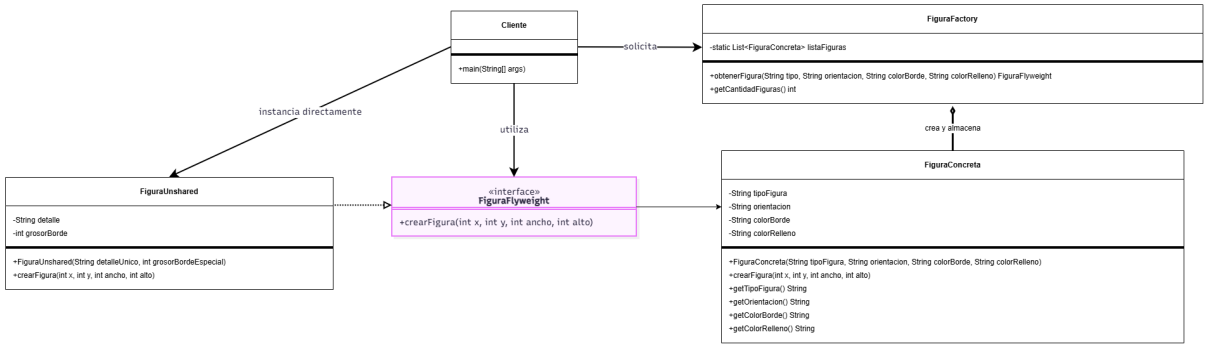


Figura 1: Diagrama de Clases del Patrón Flyweight.

Explicación de las Clases

Interfaz Flyweight (FiguraFlyweight.java)

Descripción

Declara la interfaz común para todas las figuras geométricas. Recibe como parámetros el estado extrínseco necesario para representar la figura en un contexto específico (coordenadas `x`, `y`, `ancho` y `alto`).

```java
package figuras.flyweight;

public interface FiguraFlyweight {
    void crearFigura(int x, int y, int ancho, int alto);
}
```

Clase Concreta Compartida (FiguraConcreta.java)

Descripción

Implementa la interfaz `FiguraFlyweight` y almacena exclusivamente el estado intrínseco: `tipoFigura`, `orientacion`, `colorBorde` y `colorRelleno`. Estos valores son totalmente independientes del lugar de la pantalla donde se dibuje la figura.

```java
package figuras.flyweight;

public class FiguraConcreta implements FiguraFlyweight {

    private String tipoFigura;
    private String orientacion;
    private String colorBorde;
    private String colorRelleno;

    public FiguraConcreta(String tipoFigura, String orientacion,
       String colorBorde, String colorRelleno) {
        this.tipoFigura = tipoFigura;
        this.orientacion = orientacion;
        this.colorBorde = colorBorde;
        this.colorRelleno = colorRelleno;
    }

    @Override
    public void crearFigura(int x, int y, int ancho, int alto) {
        System.out.println("Pintando figura [" + tipoFigura + ", "
           " + orientacion + ", " + colorRelleno +
                           "] en la celda con coordenadas X:" + x
                               + ", Y:" + y + " ancho: " + ancho +
                               " alto: " + alto);
    }

    public String getTipoFigura() { return tipoFigura; }
    public String getOrientacion() { return orientacion; }
```

```
25      public String getColorBorde() { return colorBorde; }
26      public String getColorRelleno() { return colorRelleno; }
27
28      public void setTipoFigura(String tipoFigura) { this.
           tipoFigura = tipoFigura; }
29      public void setOrientacion(String orientacion) { this.
           orientacion = orientacion; }
30      public void setColorBorde(String colorBorde) { this.
           colorBorde = colorBorde; }
31      public void setColorRelleno(String colorRelleno) { this.
           colorRelleno = colorRelleno; }
32  }
```

Fábrica de Flyweights (FiguraFactory.java)

Descripción

Gestiona el repositorio de objetos compartidos utilizando una lista interna. Al recibir una petición, verifica si ya existe una instancia con los mismos atributos intrínsecos; si es así, la reutiliza, y si no, crea una nueva, la almacena y la retorna.

```
1  package figuras.flyweight;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  public class FiguraFactory {
7
8      private static List<FiguraConcreta> listaFiguras = new
           ArrayList<>();
9
10     public static FiguraFlyweight obtenerFigura(String tipo,
           String orientacion, String colorBorde, String colorRelleno)
            {
11         for (FiguraConcreta fig : listaFiguras) {
12             if (fig.getTipoFigura().equals(tipo) &&
13                 fig.getOrientacion().equals(orientacion) &&
14                 fig.getColorBorde().equals(colorBorde) &&
15                 fig.getColorRelleno().equals(colorRelleno)) {
16                 return fig;
17             }
18         }
19         FiguraConcreta nuevaFigura = new FiguraConcreta(tipo,
               orientacion, colorBorde, colorRelleno);
20         listaFiguras.add(nuevaFigura);
21         return nuevaFigura;
22     }
23
24     public static int getCantidadFiguras() {
25         return listaFiguras.size();
26     }
27 }
```

Clase No Compartida (FiguraUnshared.java)

Descripción

Implementa la misma interfaz, pero representa componentes complejos o con estado único que no se benefician de la compartición en la fábrica. Se instancia de forma directa mediante el operador `new`.

```
1  package figuras.flyweight;
2
3  public class FiguraUnshared implements FiguraFlyweight {
4
5      private String detalle;
```

```
6       private int grosorBorde;
7
8       public FiguraUnshared(String detalleUnico, int
            grosorBordeEspecial) {
9           this.detalle = detalleUnico;
10          this.grosorBorde = grosorBordeEspecial;
11      }
12
13      @Override
14      public void crearFigura(int x, int y, int ancho, int alto) {
15          System.out.println("Dibujando figura NO compartida en ("
                + x + ", " + y + ")");
16      }
17  }
```

Código de Cliente

Clase Principal (Cliente.java)

Descripción

Actúa como punto de entrada de la aplicación. Solicita las figuras compartidas a la fábrica combinando múltiples escenarios y añade una figura no compartida, imprimiendo al final la estadística real de optimización de memoria.

```
1  package figuras.flyweight;
2
3  public class Cliente {
4      public static void main(String[] args) {
5          int ancho = 5;
6          int alto = 10;
7
8          FiguraFlyweight f1 = FiguraFactory.obtenerFigura("Circulo
               ", "Vertical", "Morado", "Amarillo");
9          f1.crearFigura(0, 0, ancho, alto);
10
11         FiguraFlyweight f2 = FiguraFactory.obtenerFigura("Circulo
               ", "Vertical", "Morado", "Morado");
12         f2.crearFigura(50, 0, ancho, alto);
13
14         FiguraFlyweight f3 = FiguraFactory.obtenerFigura("Rombo",
               "Horizontal", "Amarillo", "Morado");
15         f3.crearFigura(0, 50, ancho, alto);
16
17         FiguraFlyweight f4 = FiguraFactory.obtenerFigura("Rombo",
               "Horizontal", "Amarillo", "Morado");
18         f4.crearFigura(50, 50, ancho, alto);
19
20         FiguraFlyweight f5 = FiguraFactory.obtenerFigura("Circulo
               ", "Horizontal", "Morado", "Amarillo");
21         f5.crearFigura(100, 0, ancho, alto);
22
23         FiguraFlyweight f6 = FiguraFactory.obtenerFigura("Circulo
               ", "Horizontal", "Morado", "Amarillo");
24         f6.crearFigura(150, 0, ancho, alto);
25
26         FiguraFlyweight f7 = FiguraFactory.obtenerFigura("Rombo",
               "Vertical", "Morado", "Morado");
27         f7.crearFigura(100, 50, ancho, alto);
28
29         FiguraFlyweight f8 = FiguraFactory.obtenerFigura("Rombo",
               "Vertical", "Morado", "Morado");
30         f8.crearFigura(150, 50, ancho, alto);
31
32         FiguraFlyweight figuraEspecial = new FiguraUnshared("
               Borde␣con␣dise o␣ nico ", 4);
33         figuraEspecial.crearFigura(100, 100, ancho, alto);
34
```

```
35          System.out.println("\n
               ----------------------------------------");
36          System.out.println("Objetos␣unicos:␣1");
37          System.out.println("Objetos␣compartidos␣concretos:␣8");
38          System.out.println("Objetos␣reales␣concretos␣creados␣en␣
               memoria:␣" + FiguraFactory.getCantidadFiguras());
39          System.out.println("
               ----------------------------------------");
40      }
41  }
```

Pruebas y Ejecución

Ejecución en Consola

Salida obtenida en consola al ejecutar la clase `Cliente`, mostrando la reutilización de instancias compartidas y la separación del estado extrínseco.

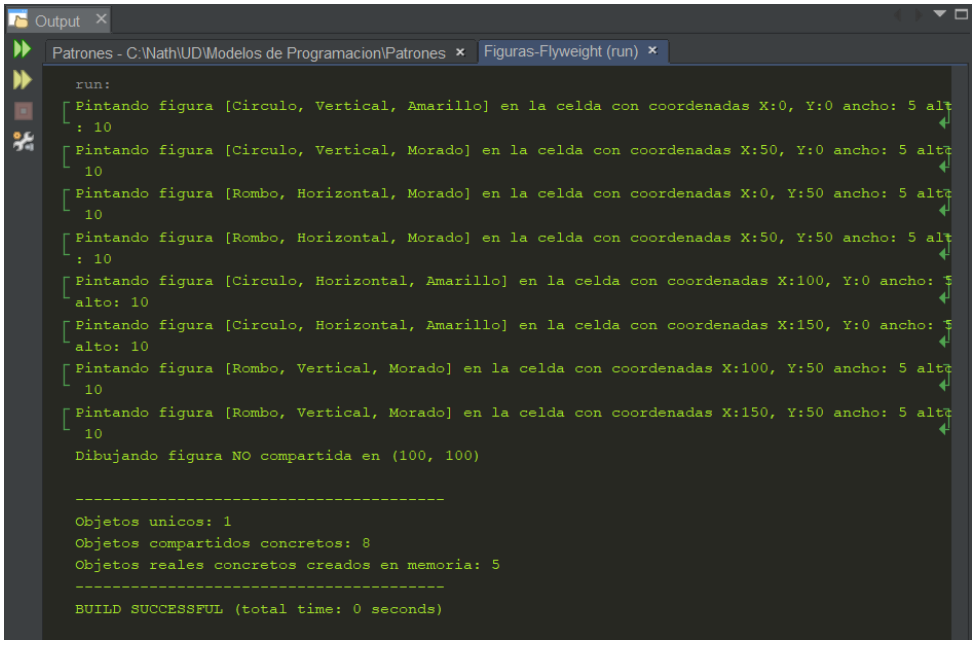


Figura 2: Captura de pantalla de la ejecución en consola del Patrón Flyweight.

Repositorio

Enlace GitHub

- https://github.com/Thalyhx/Modelos-de-Programacion/tree/main/Patrones%20Estructura/Figuras-Flyweight